

**ANO
2024**



UNINTER

ATIVIDADE PRÁTICA
ESTRUTURA DE DADOS /
PROGRAMAÇÃO III

ESTHER MARA BALDUÍNO DE PAULO RU:

Prof. Vinicius Borin, Me.

QUESTÃO 1 de 2 – Lista Encadeada

Enunciado: Com a finalidade de melhorar o atendimento e priorizar os casos mais urgentes, a direção de um hospital criou um sistema de triagem em que um profissional da saúde classifica a ordem de atendimento com base numa avaliação prévia do paciente, entregando-lhe um cartão numerado verde (V) ou amarelo (A), que define o menor ou maior grau de urgência da ocorrência, respectivamente. Para informatizar esse processo, a direção do hospital contratou você para desenvolver uma fila de chamada seguindo as seguintes regras:

- Pacientes com cartão numerado amarelo (A) são chamados antes dos pacientes com cartão numerado verde (V)
- Entre os pacientes com cartão numerado amarelo (A), os que tem numeração menor são atendidos antes.
- Entre os pacientes com cartão numerado verde (V), os que tem numeração menor são atendidos antes.
- As numerações dos cartões amarelos (A) iniciam em 201.
- As numerações dos cartões verdes (V) inicial em 1 e vão até 200.

Elabore um programa em Python que:

- A. Deve-se implementar uma **Lista Encadeada Simples** em que:
[EXIGÊNCIA DE CÓDIGO 1 de 7];
 - a. O Nodo representa um cartão numerado contendo: **número**, **cor** e um ponteiro para o **próximo**;
 - b. A lista é não circular, ou seja, seu último elemento aponta para nulo;
- B. Deve-se implementar a função **inserirSemPrioridade(nodo)** em que:
[EXIGÊNCIA DE CÓDIGO 2 de 7];

- a. Deve-se andar pela lista a partir da cabeça (**head**) e inserir o nodo no **final da lista**.
- C. Deve-se implementar a função **inserirComPrioridade(nodo)** em que: [EXIGÊNCIA DE CÓDIGO 3 de 7];
 - a. Deve-se andar pela lista a partir da cabeça (**head**) e inserir o nodo **após todos os nodos com cor "A" que estão na lista**.
 - b. O nodo inserido deve **sempre** estar antes de todos os nodos com cor "V".
- D. Deve-se implementar a função **inserir()** em que: [EXIGÊNCIA DE CÓDIGO 4 de 7];
 - a. Deve-se solicitar ao usuário a cor ("A" ou "V").
 - b. A partir da cor, o número (inteiro) do paciente deve ser **atribuído automaticamente** seguindo a ordem numérica. Por exemplo: o primeiro paciente "V" será o 201, o segundo 202, e assim por diante.
 - c. Deve-se criar um nodo com a cor e o número fornecidos pelo usuário.
 - d. Se a lista estiver vazia, a cabeça (**head**) da lista deve apontar para o nodo criado. Senão, se a cor do nodo for "V", deve-se chamar a função **inserirSemPrioridade(nodo)**. Senão, se a cor do nodo for "A", deve-se chamar a função **inserirComPrioridade(nodo)**.
- E. Deve-se implementar a função **imprimirListaEspera()** em que: [EXIGÊNCIA DE CÓDIGO 5 de 7];
 - a. Deve-se imprimir todos os cartões e seus respectivos números a partir do primeiro até o último da lista.
- F. Deve-se implementar a função **atenderPaciente()** em que: [EXIGÊNCIA DE CÓDIGO 6 de 7];
 - a. Deve-se remover o primeiro paciente da fila e imprimir uma mensagem chamando o paciente para atendimento informando o número do seu cartão.
- G. Deve-se implementar um menu para utilização do sistema em que: [EXIGÊNCIA DE CÓDIGO 7 de 7];
 - a. Deve-se apresentar as opções (1 – adicionar paciente a fila, 2 – mostrar pacientes na fila, 3 – chamar paciente, 4 – sair)
 - b. Se escolhida a opção 1, chamar a função **inserir()**.
 - c. Se escolhida a opção 2, chamar a função **imprimirListaEspera()**.
 - d. Se escolhida a opção 3, chamar a função **atenderPaciente()**.
 - e. Se escolhida a opção 4, encerrar o programa.
 - f. Se escolhida uma opção diferente as opções disponíveis, volte para o menu.

Para testar o software, execute os seguintes passos e apresente a saída do console conforme exemplo de saída de console (próxima página):

- H. Deve-se testar o sistema inserindo três (3) pacientes com cartão de cor "V", dois (2) pacientes com cartão de cor "A", dois (2) pacientes com cartão "V" e três (3) pacientes com cartão de cor "A", **nessa respectiva ordem**. [EXIGÊNCIA DE SAÍDA DE CONSOLE 1 de 3];

- I. Deve-se apresentar na saída de console a impressão da lista de espera (opção 2 do menu principal). [EXIGÊNCIA DE SAÍDA DE CONSOLE 2 de 3];
- J. Deve-se apresentar na saída de console o atendimento de dois (2) pacientes (opção 3 do menu principal) e em seguida mostrar a lista de espera (opção 2 do menu principal). [EXIGÊNCIA DE SAÍDA DE CONSOLE 3 de 3];

Apresentação de **Código da Questão 1:**

```
# Definindo a classe Nodo que representa um paciente na fila de espera.
class Nodo:
    def __init__(self, numero, cor):
        # O número do cartão do paciente.
        self.numero = numero
        # A cor do cartão do paciente (A para amarelo, V para verde).
        self.cor = cor
        # Referência para o próximo paciente na fila.
        self.proximo = None

# Definindo a classe ListaEncadeada que representa a lista de espera
dos pacientes.
class ListaEncadeada:
    def __init__(self):
        # Inicializa a lista de espera com a cabeça (head) como None
        (vazia).
        self.head = None

        # Método para inserir um paciente na lista sem considerar a
        prioridade.
    def inserirSemPrioridade(self, nodo):
        # Se a lista está vazia, o novo paciente será o primeiro.
        if self.head is None:
            self.head = nodo
        else:
            # Caso contrário, percorre a lista até o final para
            adicionar o novo paciente.
            atual = self.head
            while atual.proximo:
                atual = atual.proximo
```

```

        atual.proximo = nodo

    # Método para inserir um paciente na lista considerando a
prioridade.
    def inserirComPrioridade(self, nodo):
        # Se a lista está vazia, o novo paciente será o primeiro.
        if self.head is None:
            self.head = nodo
            # Se o primeiro paciente tem cartão verde, insere o novo
paciente antes dele.
        elif self.head.cor == "V":
            nodo.proximo = self.head
            self.head = nodo
        else:
            # Caso contrário, percorre a lista até encontrar um
paciente com cartão amarelo.
            atual = self.head
            while atual.proximo and atual.proximo.cor == "A":
                atual = atual.proximo
            nodo.proximo = atual.proximo
            atual.proximo = nodo

    # Método para inserir um paciente na lista com base na cor do
cartão.
    def inserir(self):
        # Pede a cor do cartão e garante que é 'A' ou 'V'.
        cor = input("Digite a cor do cartão do paciente (A ou V):
").strip().upper()
        if cor not in ["A", "V"]:
            print("Cor inválida! Por favor, digite 'A' ou 'V'.")
            return

        try:
            # Pede o número do cartão e verifica se é um inteiro
válido.
            numero = int(input(f"Digite o número do cartão para o
paciente de cor {cor}: "))
            # Verifica se o número está dentro dos intervalos válidos.
            if cor == "V" and (numero < 1 or numero > 200):
                print("Número inválido! Para cartões verdes, o número
deve estar entre 1 e 200.")
                return
            elif cor == "A" and numero < 201:

```

```

        print("Número inválido! Para cartões amarelos, o número
deve ser 201 ou maior.")
        return
    except ValueError:
        print("Entrada inválida! Por favor, insira um número
inteiro válido.")
        return

    # Cria um novo nodo com o número e a cor do paciente.
    novo_nodo = Nodo(numero, cor)
    # Adiciona o paciente à lista com ou sem prioridade, dependendo
da cor.
    if self.head is None:
        self.head = novo_nodo
    elif cor == "V":
        self.inserirSemPrioridade(novo_nodo)
    elif cor == "A":
        self.inserirComPrioridade(novo_nodo)

# Método para imprimir a lista de espera.
def imprimirListaEspera(self):
    atual = self.head
    if atual is None:
        print("A lista de espera está vazia.")
    else:
        # Cria uma lista para armazenar as informações dos
pacientes.
        lista_pacientes = []
        while atual:
            # Adiciona a informação do paciente na lista.
            lista_pacientes.append(f"[{atual.cor},{atual.numero}]")
            atual = atual.proximo
        # Imprime a lista de espera formatada.
        print("Lista de espera -> " + " ".join(lista_pacientes))

# Método para atender (remover) o próximo paciente da fila.
def atenderPaciente(self):
    if self.head is None:
        print("Não há pacientes na fila para atendimento.")
    else:
        # O primeiro paciente da lista é o próximo a ser atendido.
        paciente = self.head

```

```

        print(f"Chamando paciente com cartão
{paciente.cor}{paciente.numero} para atendimento.")
        # Atualiza a cabeça da lista para o próximo paciente.
        self.head = self.head.proximo

# Função principal para exibir o menu e permitir a interação com o
usuário.
def menu():
    lista_espera = ListaEncadeada()
    while True:
        print("\nMenu:")
        print("1 - Adicionar paciente à fila")
        print("2 - Mostrar pacientes na fila")
        print("3 - Chamar paciente")
        print("4 - Sair")
        opcao = input("Escolha uma opção: ")

        if opcao == "1":
            lista_espera.inserir()
        elif opcao == "2":
            lista_espera.imprimirListaEspera()
        elif opcao == "3":
            lista_espera.atenderPaciente()
        elif opcao == "4":
            break
        else:
            print("Opção inválida. Tente novamente.")

# Executa o menu principal quando o script é executado.
if __name__ == "__main__":
    menu()

```

Apresentação de *Saída do Console da Questão 1:*

Menu:

- 1 - Adicionar paciente à fila
- 2 - Mostrar pacientes na fila
- 3 - Chamar paciente
- 4 - Sair

Escolha uma opção: 1

Digite a cor do cartão do paciente (A ou V): V

Digite o número do cartão para o paciente de cor V: 40

Menu:

- 1 - Adicionar paciente à fila
- 2 - Mostrar pacientes na fila
- 3 - Chamar paciente
- 4 - Sair

Escolha uma opção: 1

Digite a cor do cartão do paciente (A ou V): V

Digite o número do cartão para o paciente de cor V: 20

Menu:

- 1 - Adicionar paciente à fila
- 2 - Mostrar pacientes na fila
- 3 - Chamar paciente
- 4 - Sair

Escolha uma opção: 1

Digite a cor do cartão do paciente (A ou V): V

Digite o número do cartão para o paciente de cor V: 2

Menu:

- 1 - Adicionar paciente à fila
- 2 - Mostrar pacientes na fila
- 3 - Chamar paciente
- 4 - Sair

Escolha uma opção: 1

Digite a cor do cartão do paciente (A ou V): A

Digite o número do cartão para o paciente de cor A: 204

Menu:

- 1 - Adicionar paciente à fila
- 2 - Mostrar pacientes na fila
- 3 - Chamar paciente
- 4 - Sair

Escolha uma opção: 1

Digite a cor do cartão do paciente (A ou V): A

Digite o número do cartão para o paciente de cor A: 255

Menu:

- 1 - Adicionar paciente à fila
- 2 - Mostrar pacientes na fila
- 3 - Chamar paciente
- 4 - Sair

Escolha uma opção: 1

Digite a cor do cartão do paciente (A ou V): V

Digite o número do cartão para o paciente de cor V: 33

```
Menu:
1 - Adicionar paciente à fila
2 - Mostrar pacientes na fila
3 - Chamar paciente
4 - Sair
Escolha uma opção: 1
Digite a cor do cartão do paciente (A ou V): V
Digite o número do cartão para o paciente de cor V: 90
```

```
Menu:
1 - Adicionar paciente à fila
2 - Mostrar pacientes na fila
3 - Chamar paciente
4 - Sair
Escolha uma opção: 1
Digite a cor do cartão do paciente (A ou V): A
Digite o número do cartão para o paciente de cor A: 300
```

```
Menu:
1 - Adicionar paciente à fila
2 - Mostrar pacientes na fila
3 - Chamar paciente
4 - Sair
Escolha uma opção: 1
Digite a cor do cartão do paciente (A ou V): A
Digite o número do cartão para o paciente de cor A: 270
```

```
Menu:
1 - Adicionar paciente à fila
2 - Mostrar pacientes na fila
3 - Chamar paciente
4 - Sair
Escolha uma opção: 1
Digite a cor do cartão do paciente (A ou V): A
Digite o número do cartão para o paciente de cor A: 220
```

```
Menu:
1 - Adicionar paciente à fila
2 - Mostrar pacientes na fila
3 - Chamar paciente
4 - Sair
Escolha uma opção: 2
Lista de espera -> [A,204] [A,255] [A,300] [A,270] [A,220] [V,40] [V,20] [V,2] [V,33] [V,90]
```

```
Menu:
1 - Adicionar paciente à fila
2 - Mostrar pacientes na fila
3 - Chamar paciente
4 - Sair
Escolha uma opção: 3
Chamando paciente com cartão A204 para atendimento.
```

```
Menu:
1 - Adicionar paciente à fila
2 - Mostrar pacientes na fila
3 - Chamar paciente
4 - Sair
Escolha uma opção: 3
Chamando paciente com cartão A255 para atendimento.
```

```
Menu:
1 - Adicionar paciente à fila
2 - Mostrar pacientes na fila
3 - Chamar paciente
4 - Sair
Escolha uma opção: 2
Lista de espera -> [A,300] [A,270] [A,220] [V,40] [V,20] [V,2] [V,33] [V,90]
```

QUESTÃO 2 de 2 – Tabela Hash

Enunciado: Com o objetivo de criar um sistema novo de emplacamento de veículos, deputados em do Distrito Federal – DF, decidiram que o último número da placa dos veículos, irá representar o estado de registro dele. Para isso, sua equipe de desenvolvedores foi encarregada de desenvolver uma **Tabela Hash com endereçamento em cadeia de 10 posições** (cada posição do vetor deve ser uma lista encadeada), representando os números de 0 a 9 que irão representar os 26 estados e o Distrito Federal (total 27).

A função hash deve seguir as seguintes regras:

- A entrada da função hash deve ser uma string com 2 letras, representando a sigla do estado e/ou distrito federal.
- Caso a sigla seja DF (Distrito Federal), por questões de superstição, os deputados solicitaram que o retorno da função seja 7 sempre.
- Caso contrário, a função deve retornar a posição com base no valor ASCII das duas letras e seguindo a seguinte regra:

$$posição = (CHAR1_{ASCII} + CHAR2_{ASCII}) MOD 10$$

Onde $CHAR1_{ASCII}$ e $CHAR2_{ASCII}$ são os valores ASCII da primeira e segunda letra, respectivamente (Tabela ASCII no final do documento).

Elabore um programa em Python que:

- A. Deve-se implementar a tabela Hash com 10 posições, onde inicialmente todas as posições possuem valor **None** [EXIGÊNCIA DE CÓDIGO 1 de 7];
- B. Deve-se implementar as **Listas Encadeadas Simples** em que: [EXIGÊNCIA DE CÓDIGO 2 de 7];
 - a. O Nodo representa um Estado contendo: **sigla**, **nomeEstado** e um ponteiro para o **próximo**;
 - b. As 10 posições da tabela hash, representam a cabeça de cada lista (**head**).
- C. Deve-se implementar a inserção no início da lista encadeada (cada elemento novo deve ser sempre **inserido no início da lista**) [EXIGÊNCIA DE CÓDIGO 3 de 7];
- D. Deve-se implementar a impressão da tabela hash, onde devem ser impressas as **siglas** de todos os nodos que estão na tabela hash **separados por posição** [EXIGÊNCIA DE CÓDIGO 4 de 7];
- E. Deve-se implementar a função hash, conforme enunciado. [EXIGÊNCIA DE CÓDIGO 5 de 7];
- F. Deve-se implementar a inserção dos estados e distrito federal (**todos os 27 com nome e sigla**) na tabela hash utilizando a função hash (não precisa solicitar ao usuário a digitação via teclado, pode inserir no código mesmo de modo *hard code*) [EXIGÊNCIA DE CÓDIGO 6 de 7];
- G. Deve-se inserir na Tabela, além dos estados e distrito federal, um estado fictício, sendo que esse estado tenha seu **nome completo** e como siglas, a primeira letra do seu nome e a primeira letra do seu último sobrenome. Exemplo: Bruno Kostiuk – BK. [EXIGÊNCIA DE CÓDIGO 7 de 7];

Para testar o software, execute os seguintes passos e apresente a saída do console conforme exemplo de saída de console (próxima página):

- H. Deve-se apresentar na saída de console, a impressão da tabela hash antes de inserir qualquer informação [EXIGÊNCIA DE SAÍDA DE CONSOLE 1 de 3];
- I. Deve-se apresentar na saída de console, a impressão da tabela hash após inserir os 26 estados e o Distrito Federal - DF [EXIGÊNCIA DE SAÍDA DE CONSOLE 2 de 3];
- J. Deve-se apresentar na saída de console, a impressão da tabela hash após inserir os 26 estados, Distrito Federal – DF e o estado fictício com seu nome completo. [EXIGÊNCIA DE SAÍDA DE CONSOLE 3 de 3];

Caracteres de controle ASCII			ASCII caracteres imprimíveis			Caracteres ASCII estendidos		
00	NULL	(Null character)	32	space	64	@	96	`
01	SOH	(Start of Header)	33	!	65	A	97	a
02	STX	(Start of Text)	34	"	66	B	98	b
03	ETX	(End of Text)	35	#	67	C	99	c
04	EOT	(End of Trans.)	36	\$	68	D	100	d
05	ENQ	(Enquiry)	37	%	69	E	101	e
06	ACK	(Acknowledgement)	38	&	70	F	102	f
07	BEL	(Bell)	39	'	71	G	103	g
08	BS	(Backspace)	40	(	72	H	104	h
09	HT	(Horizontal Tab)	41	)	73	I	105	i
10	LF	(Line feed)	42	*	74	J	106	j
11	VT	(Vertical Tab)	43	+	75	K	107	k
12	FF	(Form feed)	44	,	76	L	108	l
13	CR	(Carriage return)	45	-	77	M	109	m
14	SO	(Shift Out)	46	.	78	N	110	n
15	SI	(Shift In)	47	/	79	O	111	o
16	DLE	(Data link escape)	48	0	80	P	112	p
17	DC1	(Device control 1)	49	1	81	Q	113	q
18	DC2	(Device control 2)	50	2	82	R	114	r
19	DC3	(Device control 3)	51	3	83	S	115	s
20	DC4	(Device control 4)	52	4	84	T	116	t
21	NAK	(Negative acknowl.)	53	5	85	U	117	u
22	SYN	(Synchronous idle)	54	6	86	V	118	v
23	ETB	(End of trans. block)	55	7	87	W	119	w
24	CAN	(Cancel)	56	8	88	X	120	x
25	EM	(End of medium)	57	9	89	Y	121	y
26	SUB	(Substitute)	58	:	90	Z	122	z
27	ESC	(Escape)	59	;	91	[	123	{
28	FS	(File separator)	60	<	92	\	124	
29	GS	(Group separator)	61	=	93	]	125	}
30	RS	(Record separator)	62	>	94	^	126	~
31	US	(Unit separator)	63	?	95	_		
127	DEL	(Delete)						
128	Ç		160	á	192	Ł	224	ó
129	ü		161	í	193	ł	225	ß
130	é		162	ó	194	Ł	226	Ô
131	â		163	ú	195	ł	227	Õ
132	ä		164	ñ	196	—	228	ö
133	à		165	Ñ	197	†	229	Ó
134	á		166	ª	198	ä	230	µ
135	ç		167	º	199	Å	231	þ
136	ê		168	¿	200	Ł	232	þ
137	ë		169	®	201	Œ	233	Û
138	è		170	™	202	Œ	234	Ü
139	ï		171	½	203	Œ	235	Ù
140	î		172	¼	204	Œ	236	Ý
141	ì		173	ì	205	=	237	Ÿ
142	Ä		174	«	206	†	238	—
143	Å		175	»	207	‡	239	'
144	É		176	»	208	ø	240	≡
145	æ		177	»	209	Ð	241	±
146	Æ		178	»	210	Ê	242	¾
147	ø		179	»	211	Ê	243	¾
148	ö		180	»	212	Ê	244	¶
149	ò		181	À	213	ì	245	§
150	û		182	À	214	í	246	÷
151	ù		183	À	215	î	247	°
152	ÿ		184	©	216	ÿ	248	°
153	Ö		185	»	217	»	249	°
154	Ü		186	»	218	»	250	°
155	ø		187	»	219	»	251	°
156	£		188	»	220	»	252	°
157	Ø		189	¢	221	»	253	°
158	×		190	¥	222	»	254	°
159	f		191	»	223	»	255	nbsp

Apresentação de **Código da Questão 2:**

```
# Definindo a classe Node que representa um estado na lista encadeada.
class Node:
    def __init__(self, sigla, nomeEstado):
        # Sigla do estado (abreviação de duas letras).
        self.sigla = sigla
        # Nome completo do estado.
        self.nomeEstado = nomeEstado
        # Referência para o próximo nó na lista.
        self.next = None
```

```
# Definindo a classe LinkedList para gerenciar uma lista encadeada de estados.
```

```
class LinkedList:
```

```
    def __init__(self):
```

```
        # Inicializa a lista com a cabeça (head) como None (vazia).
```

```
        self.head = None
```

```
    # Método para inserir um novo estado no início da lista.
```

```
    def insert_at_beginning(self, sigla, nomeEstado):
```

```
        new_node = Node(sigla, nomeEstado)
```

```
        # Faz o novo nó apontar para a cabeça atual da lista.
```

```
        new_node.next = self.head
```

```
        # Atualiza a cabeça da lista para o novo nó.
```

```
        self.head = new_node
```

```
    # Método para imprimir a lista de estados.
```

```
    def print_list(self):
```

```
        current = self.head
```

```
        if not current:
```

```
            print("None") # Imprime "None" se a lista estiver vazia.
```

```
        else:
```

```
            while current:
```

```
                # Imprime a sigla do estado, seguido por uma seta.
```

```
                print(current.sigla, end=" -> ")
```

```
                # Move para o próximo nó na lista.
```

```
                current = current.next
```

```
            print("None") # Indica o final da lista.
```

```
# Definindo a classe HashTable para gerenciar uma tabela hash de listas encadeadas.
```

```
class HashTable:
```

```
    def __init__(self):
```

```
        # Define o tamanho da tabela hash.
```

```
        self.size = 10
```

```
        # Cria a tabela como uma lista de LinkedLists.
```

```
        self.table = [LinkedList() for _ in range(self.size)]
```

```
    # Função hash para determinar o índice da tabela para uma sigla de estado.
```

```
    def hash_function(self, sigla):
```

```
        if sigla == "DF":
```

```

        return 7 # Força o Distrito Federal para uma posição
específica.

    char1_ascii = ord(sigla[0])
    char2_ascii = ord(sigla[1])

    # Calcula o índice usando a soma dos códigos ASCII dos
caracteres da sigla.
    return (char1_ascii + char2_ascii) % self.size

# Método para inserir um estado na tabela hash.
def insert(self, sigla, nomeEstado):
    index = self.hash_function(sigla)
    # Insere o estado na lista encadeada correspondente ao índice
calculado.
    self.table[index].insert_at_beginning(sigla, nomeEstado)

# Método para imprimir a tabela hash.
def print_table(self):
    for i in range(self.size):
        print(f"Posição {i}: ", end="")
        self.table[i].print_list()

# Função para adicionar todos os estados e o Distrito Federal na tabela
hash.
def add_all_states_to_table(hash_table):
    states = [
        ("AC", "Acre"), ("AL", "Alagoas"), ("AM", "Amazonas"), ("AP",
"Amapá"), ("BA", "Bahia"),
        ("CE", "Ceará"), ("DF", "Distrito Federal"), ("ES", "Espírito
Santo"), ("GO", "Goiás"),
        ("MA", "Maranhão"), ("MG", "Minas Gerais"), ("MS", "Mato Grosso
do Sul"), ("MT", "Mato Grosso"),
        ("PA", "Pará"), ("PB", "Paraíba"), ("PE", "Pernambuco"), ("PI",
"Piauí"), ("PR", "Paraná"),
        ("RJ", "Rio de Janeiro"), ("RN", "Rio Grande do Norte"), ("RO",
"Rondônia"), ("RR", "Roraima"),
        ("RS", "Rio Grande do Sul"), ("SC", "Santa Catarina"), ("SE",
"Sergipe"), ("SP", "São Paulo"),
        ("TO", "Tocantins")
    ]
    # Adiciona todos os estados e o Distrito Federal na tabela hash.
    for sigla, nome in states:
        hash_table.insert(sigla, nome)

```

```
# Função principal para executar o programa.
def main():
    hash_table = HashTable()

    # Impressão da tabela hash antes de inserir qualquer informação.
    print()
    hash_table.print_table()
    print()

    # Adiciona todos os estados e o Distrito Federal na tabela hash.
    add_all_states_to_table(hash_table)

    # Impressão da tabela hash após inserir os 26 estados e o Distrito
Federal.
    print()
    hash_table.print_table()
    print()

    # Insere um estado fictício na tabela hash.
    hash_table.insert("EP", "Esther Mara Balduino de Paulo")

    # Impressão da tabela hash após inserir os 26 estados, o Distrito
Federal e o estado fictício.
    print()
    hash_table.print_table()

# Executa a função principal quando o script é executado.
if __name__ == "__main__":
    main()
```

Apresentação de **Saída do Console da Questão 2:**

```
Posição 0: None
Posição 1: None
Posição 2: None
Posição 3: None
Posição 4: None
Posição 5: None
Posição 6: None
Posição 7: None
Posição 8: None
Posição 9: None

Posição 0: SC -> RN -> MS -> GO -> None
Posição 1: RO -> MT -> BA -> AL -> None
Posição 2: SE -> PR -> MA -> ES -> AM -> AC -> None
Posição 3: TO -> SP -> PI -> None
Posição 4: RR -> None
Posição 5: RS -> PA -> AP -> None
Posição 6: RJ -> PB -> CE -> None
Posição 7: DF -> None
Posição 8: MG -> None
Posição 9: PE -> None
```

```
Posição 0: SC -> RN -> MS -> GO -> None
Posição 1: RO -> MT -> BA -> AL -> None
Posição 2: SE -> PR -> MA -> ES -> AM -> AC -> None
Posição 3: TO -> SP -> PI -> None
Posição 4: RR -> None
Posição 5: RS -> PA -> AP -> None
Posição 6: RJ -> PB -> CE -> None
Posição 7: DF -> None
Posição 8: MG -> None
Posição 9: EP -> PE -> None
```